

vLLM-Ascend 原生不支持 MoE Expert Offload 推理

按 MoE 推理执行流程拆解证据与卡点

2026 年 6 月 11 日

结论先行： 官方/原生 vLLM-Ascend 支持 MoE 推理、EP/EPLB、KV cache CPU offload 和 cache/L2 weight prefetch，但不提供“按当前 routed experts 从 CPU/host 侧加载 expert 权重到 NPU 固定槽位再执行”的 MoE expert offload runtime。

范围界定

本文讨论的 “MoE offload”

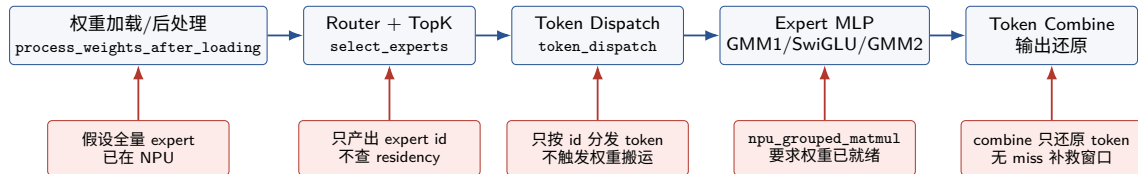
- Expert 权重不全量常驻 HBM。
- 每轮根据 `topk_ids` 得到 routed expert working set。
- 缺失 expert 从 CPU/host store 搬到 NPU resident slot。
- `npu_grouped_matmul` 只看到 device-resident、layout-compatible 的权重。
- 需要命中/缺失、替换、预取、同步和执行分阶段。

容易混淆但不是它

- **KV cache CPU offload**: 搬 KV blocks, 不搬模型 expert 权重。
- **Ascend weight prefetch**: 把已在设备侧的权重预取到 L2/cache, 不是 host-to-HBM。
- **vLLM 通用 prefetch offload**: 按 layer/parameter hook, 不感知 MoE routed expert 集合。
- **research 分支原型**: 是为补这个缺口而加的实验能力, 不代表官方原生能力。

对照基线: 官方参考仓库 `/workspace/reference-repos/vllm-ascend@7c4ec8e5 main` 中未检索到 `ascend-moe-offload`、`fixed_slot`、`moe_offload_runtime` 等原生入口; 本地 `research@a02ed84a1` 已出现这些原型代码。

MoE 推理流程总览



核心断点：原生流程中，唯一真正消耗 expert 权重的是 MLP grouped matmul；在此之前的 routing/dispatch 有 expert id 和 token 计数，却没有“CPU store → NPU slot”的状态机。

Step 0: 权重加载与后处理

源码证据

- official:
vllm_ascend/ops/fused_moe/fused_moe.py:107-127
- w13_weight/w2_weight 在加载后被 pad、transpose、contiguous。
- 按配置转成 Ascend 需要的 NZ/ND 格式。
- 处理结果重新赋给 layer.w13_weight 和 layer.w2_weight。

为什么这里卡 offload

- 原生后处理默认“完整 expert tensor 已经存在”。
- 后续 GMM 依赖后处理后的 shape、stride、dtype、layout。
- 如果 generic offload 把参数留在 CPU，MLP 阶段拿到的就是 CPU tensor。
- 如果运行时动态新分配 device tensor，地址/layout 又不稳定，图重放和静态 kernel 不友好。

所需但缺失： CPU expert store 必须保存“后处理后”的 expert 权重；NPU 侧还要有固定 slot tensor，不能让 grouped matmul 直接消费普通 CPU 参数对象。

Step 1: Router / TopK

原生做了什么

- official: fused_moe.py:159-180
- `select_experts(hidden_states=x, router_logits=..., top_k=...)`
- 输出 `topk_weights`, `topk_ids`, 即每个 token 选中的 expert 与权重。
- 这一步只定义语义上的路由结果。

offload 需要的额外状态

- 当前 layer 的 expert residency。
- `expert_id` -> `slot_id` 映射。
- miss expert 的 host 位置、copy 事件、deadline。
- 按 token count 判断 miss 影响面。

原生缺口

- 官方 `MoEFusedExpertsInput` 没有 `offload` 字段。
- `topk_ids` 直接进入 `dispatch`, 不经过 `residency manager`。
- `enable_return_routed_experts` 只是捕获路由结果, 不负责搬权重。

Step 2: Token Dispatch / Group List

源码证据

- official: `moe_comm_method.py:138-153`
- 先记录 NPU event, 再把 `topk_ids` 交给 `token_dispatcher.token_dispatch(...)`。
- official: `token_dispatcher.py:230-260` 在 MC2 路径调用 `npu_moe_distribute_dispatch`, 返回 `expand_x` 和 `expert_token_nums/group_list`。

dispatch 的边界

- 它处理 token 到 expert 的排列、通信和计数。
- 它不检查 expert 权重是否在 NPU。
- 它不产生 host-to-HBM copy task。

为什么不能在这里自然补上

- dispatch 输出的是激活和 token metadata, 不是权重对象。
- 搬权重必须知道后续 GMM 的权重 layout 与 slot 张量。
- 如果这里只按 `topk_ids` 异步搬, MLP 前仍缺少“等待哪些 slot”的执行协议。

Step 3: Expert MLP / Grouped Matmul

源码证据

- official: moe_mlp.py:362-389
- 非量化路径直接调用两次
`torch_npu.npu_grouped_matmul`。
- `x=[hidden_states]` 与 `weight=[w1]/[w2]` 同时进入 NPU op。
- 量化路径也在 moe_mlp.py:176-217, 298-340 消费 `w1/w2`。

根因

- grouped matmul 是真正的 expert 权重消费点。
- 输入激活在 NPU；权重如果还在 CPU，会出现 device mismatch。
- grouped op 按 `group_list` 批量算 expert，不接受“这个 expert 先去加载”的回调。
- 原生路径没有把 miss expert 拆成 phase 0/phase 1。

实测断点：通用 prefetch expert-only 已能把 resident weight 从约 56.9GB 降到约 43.4GB，但 profile/generate 前在 `npu_grouped_matmul` 处报 CPU/NPU tensor mixing。

Step 4: Combine / 输出还原

源码证据

- official: `moe_comm_method.py:155-171`
- MLP 完成后记录 `before_combine_evt`。
- `token_dispatcher.token_combine(hidden_states=mlp_output, combine_metadata=...)`
- 返回 `FusedExpertsResult(routed_out=..., expert_tokens=group_list)`

为什么 combine 阶段救不了 offload

- combine 只消费 MLP 输出，语义上已经过了 expert 计算点。
- 如果某些 expert 权重未加载，MLP 根本无法完整产生对应 token 输出。
- offload 的等待、补算、分阶段执行必须发生在 MLP 前或 MLP 内部，而不是 combine 后。

原生支持了什么，但不是 Expert Offload

能力	证据	为什么不是 MoE expert offload
MoE 推理 / EP / EPLB	fused_moe.py、token_dispatcher.py、release notes 多处 MoE/EP 支持	假设 local experts 已经放在当前 rank/NPU 上；解决 routing、dispatch、combine 和负载均衡，不解决 CPU expert store 与 slot miss。
KV Cache CPU Offload	官方文档说明 offload inactive KV cache blocks，并用 NPUoffloadingSpec 做 NPU/CPU 异步传输	对象是 KV cache block，不是模型参数；触发条件是 prefix/cache block 命中缺失，不是 routed expert 集合。
Weight Prefetch	官方文档说明把权重预加载到 L2/cache，利用 MoE gating top-k、RMSNorm、SwiGLU 等 vector 计算隐藏 CMO	前提是权重已经 device-resident；代码中取 mlp.experts.w13_weight 做 cache prefetch，不是 host-to-HBM 加载。

外部文档证据： Weight Prefetch Guide 明确写的是预加载到 L2 cache；KV Cache CPU Offload Guide 明确写的是 KV cache blocks。两者都没有 expert 权重驻留/替换/slot 语义。

vLLM 通用 Weight Offload 也接不上

配置层证据

- `vllm/config/offload.py:12-89`
- backend 只有 auto, uva, prefetch。
- prefetch 配置是 group size、group 内 offload 数、prefetch step 和参数白名单。
- 文档/注释写的是 group-based layer offloading。

执行层证据

- `prefetch.py:175-205` 按 module index 选择层。
- `prefetch.py:213-233` hook module forward, 插入 wait/start prefetch。
- `prefetch.py:338-365` 分配 static buffer pool 后按 layer 顺序预取。
- 没有 `topk_ids`、expert token count、slot miss、replacement policy。

根因：通用 offload 的最小调度单元是“层/参数”；MoE expert offload 的最小调度单元是“某 layer 的某 expert working set”。调度粒度错了，哪怕省了 HBM，也无法保证 grouped matmul 前 expert 已就绪。

UVA 路径为什么不成立

源码证据

- `vllm/model_executor/offloader/uva.py:21-35` 描述为 UVA zero-copy。
- `uva.py:97-105` 将参数放 CPU，再调用 `get_accelerator_view_from_cpu_tensor(cpu_data)`。
- `vllm/utils/torch_utils.py:759-773` 对不支持的平台抛出 “not supported in: <device>”。

实测证据

- `docs/sew-offload/05-existing-offload-baseline.md`
- Qwen3-30B-A3B 单 910B3，UVA expert offload 在加载前失败。
- 报错：`get_accelerator_view_from_cpu_tensor is currently not supported in: npu`

结论： UVA 不是 Ascend MoE expert offload 的可用 baseline；即使可用，也会让 NPU grouped matmul 跨 CPU memory 取权重，和 Ascend 高性能执行路径不匹配。

Prefetch 路径为什么仍然会断

CUDA-shaped

- prefetch.py:156 使用 `torch.cuda.Stream()`。
- prefetch.py:256-273 使用 `torch.cuda.is_current_stream_capturing()`、`current_stream().wait_event()`。
- prefetch.py:517-543 用 CUDA event/stream 管理 copy。
- Ascend 生产路径应显式使用 `torch.npu stream/event`。

MoE-unaware

- 只知道层序号和参数名 `whitelist`。
- 不知道本轮 routed expert id。
- 不知道哪些 expert resident、哪些 miss。
- 不知道 `group_list` 中每个 expert 的 token 数和截止时间。

症状： generic prefetch 可以降低常驻权重，但没有把 `w13_weight/w2_weight` 替换成“已就绪的 NPU slot-backed weights”，所以到 MLP GMM 边界仍会暴露 CPU tensor 或错误同步。

实测基线：能省显存，但不能生成

Case	resident weight log	结果	关键信号
No offload	56.9001 GB	成功	Qwen3-30B-A3B 在 64GB 910B3 上仅小上下文勉强可跑，峰值约 63.9GB。
UVA expert offload	未进入加载	失败	UVA accelerator view 不支持 npu。
Prefetch expert-only	43.4001 GB	失败	显存下降约 13.5GB，但 npu_grouped_matmul 看到 weight 在 CPU、activation 在 NPU。

解释：这不是“不需要 offload”，恰恰相反：显存节省真实存在；问题是原生/通用 offload 没有 MoE execution boundary 的专家驻留协议。

为什么必须在 MoE Boundary 解决

这个 boundary 能看到的信息

- `topk_ids/topk_weights`: 本轮 routed experts。
- `layer_id`: 当前 layer deadline。
- `w13_weight/w2_weight`: 后处理后的 expert 权重对象。
- `expert_map/log2phy/global_redundant_expert_num`: EP/EPLB 映射。
- `moe_comm_type`: AllGather、MC2、Fused MC2 等通信路径。

在这个 boundary 之前/之后都不合适

- 之前: 不知道当前 routed working set, 也无法精确判断 miss。
- 之后: MLP 已经要求权重就绪, combine 阶段太晚。
- 因此 offload runtime 必须在 `select_experts -> build_fused_experts_input -> fused_experts` 之间介入。

图模式与稳定地址的额外约束

官方/原生路径的隐含假设

- 权重 tensor 地址和 layout 稳定。
- grouped matmul 的输入签名稳定。
- dispatch/MLP/combine 可以被事件和图执行串起来。
- process_weights_after_loading 后权重就地参与后续执行。

动态 expert cache 会破坏什么

- miss 时临时分配 tensor，地址不稳定。
- 每轮 expert 集合变化，kernel 输入权重对象变化。
- CPU 决策路径无法被图捕获。
- 需要固定 slots，让图只看到 slot tensor，运行时只改 expert_id -> slot_id。

research 分支旁证： README 明确说 MoE offload 原型需要 `--enforce-eager`，因为 graph capture 不能记录 offload scheduler 的 runtime CPU decision path。

Research 分支为什么是“补缺口”的证据

新增内容与官方 main 的差异

- research README: `--ascend-moe-offload-gb 14`
- research `fused_moe.py`:222-335 增加 `moe_offload_runtime.trace_routing`、`fixed slot plan`、`offload params`。
- research `moe_runtime_args.py`:149-189 增加 `MoEOffloadParams`。
- 官方 main 对应文件没有这些字段，`MoEFusedExpertsInput` 只包含 `hidden/topk/weights/routing/quant`。

原型自身暴露的未覆盖面

- `fixed slots` 当前限制 `AllGather`。
- 不支持 `expert_map`、`redundant experts`、`expert bias`、`zero expert path`。
- 当前走 `eager path`。

含义：如果官方原生已经支持 MoE expert offload，就不需要在 research 分支额外引入 runtime、slot 注册、routing trace 和 offload args。

按执行步骤汇总卡点

步骤	原生证据	缺失能力	根因
权重后处理	fused_moe.py:107-127	CPU expert store + NPU fixed slots	后处理默认全量 expert tensor 已在设备侧；offload 必须保存后处理 layout。
Router/TopK	fused_moe.py:159-180	residency lookup、miss set、deadline	topk_ids 只表示语义路由，不绑定设备驻留。
Dispatch	moe_comm_method.py:138-153； token_dispatcher.py:230-260	copy task 生成、slot wait 协议	dispatch 只处理 token permutation/communication/group list，不处理权重。
MLP GMM	moe_mlp.py:362-389	slot-backed w1/w2、phase execution	npu_grouped_matmul 一次性消费权重；CPU/NPU 混用直接失败。
Combine	moe_comm_method.py:155-171	无	combine 太晚，只还原 MLP 输出，无法补救未计算的 expert。
通用 offload	offload.py:12-89； prefetch.py:175-233	expert-aware scheduling	粒度是 layer/parameter，不是 routed expert working set。

要变成原生支持，需要补哪些件

数据面

- 后处理后的 CPU expert store。
- 每层固定 NPU expert slots。
- slot tensor 保持 dtype、shape、stride、NZ/ND layout。
- expert_id -> slot_id 映射与版本号。

控制面

- 从 topk_ids/group_list 得到 miss set 与 token impact。
- native torch.npu stream/event H2D 调度。
- deadline-aware prefetch 与 replacement。
- hit-first phased execution 或同步 slot prototype。

最小正确性要求： grouped matmul 调用点看到的永远是 NPU-resident slot weights，而不是 CPU 参数、临时 device tensor 或 layout 未后处理的权重。

证据索引

本地源码

- 官方参考: `/workspace/reference-repos/vllm-ascend@7c4ec8e5 main`
- 当前研究分支: `/workspace/vllm-ascend-hust@a02ed84a1 research`
- vLLM-HUST: `/workspace/vllm-hust`
- MoE 主链路: `vllm_ascend/ops/fused_moe/fused_moe.py`、`moe_comm_method.py`、`token_dispatcher.py`、`moe_mlp.py`
- 通用 offload: `vllm/config/offload.py`、`vllm/model_executor/offloader/prefetch.py`、`uva.py`
- 实测记录: `docs/sew-offload/05-existing-offload-baseline.md`、`08-ascend-moe-offload-architecture.md`

官方文档

- vLLM-Ascend Weight Prefetch Guide:
https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/weight_prefetch.html
- vLLM-Ascend KV Cache CPU Offload Guide:
https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/kv_cache_cpu_offload.html
- vLLM Fused MoE Modular Kernel: https://docs.vllm.ai/en/latest/design/fused_moe_modular_kernel/